## OPERATING SYSTEMS

### ASSIGNMENT 3 – CO3

**CO3 Coverage:** Evaluate memory-management mechanisms such as linking, paging and segmentation to determine their effectiveness for different system requirements – linking and loading, paging, segmentation, virtual memory, demand paging, page-replacement techniques and disk management.

**Nature of Assignment:** Individual
**Duration:** 1 Week
**Total Marks:** 50
**Submission Date:** _____

**Bloom's Levels Assessed:** K5 – Evaluate (every task in this assignment is at K5 level)

**Mandatory Execution and GitHub Submission**

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **CSA04-<RegisterNumber>-Assignment3** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: _____

**How to Answer the Additional Questions**

**Complete the Code:** Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

**Alternative Concept and Complexity Analysis:** Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

---

**Assignment Principle**

Every question asks you to design, implement, analyse and evaluate the solution yourself, so that you demonstrate evidence of:

**Design → Implementation → Analysis → Evaluation**

---

## Q1 – Linking, Loading and Address Binding — 6 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Updating a Banking Application**

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch must currently receive a new 200 MB executable.

**Implementation Tasks**

You must:

- Build the same program with static linking and with dynamic (shared) libraries
- Compare the sizes of the executables and list the shared libraries with ldd
- Load a library at run time with dlopen() and dlsym()
- Show the addresses at which the program and its libraries are loaded (/proc/self/maps)

**Evaluation**

You must:

- Explain compile-time, load-time and execution-time address binding
- Explain the roles of the linker and the loader
- Evaluate static and dynamic linking for memory use, start-up time and ease of updating
- Explain position-independent code

**Experimental Investigation**

You must:

- Measure the start-up time and memory use of both versions
- Replace the shared library without recompiling the program and show the change
- Run two copies and show which pages are shared
- Test what happens when the shared library is missing

**Conclusion**

*Which linking method should the bank use, and why?*

**Alternative Concept and Complexity Analysis**

If the bank loaded the encryption library at run time with dlopen() only when needed, instead of linking it dynamically at start-up, what would the solution be? Evaluate the start-up time, the memory used when encryption is not needed, and the error handling needed if the library is missing.

---

## Q2 – Paging — 8 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Memory for a Multi-User College Server**

A college server runs hundreds of student programs of very different sizes. Allocating each program one contiguous block of memory left many small unusable gaps, and large programs could not be loaded although enough total memory was free.

**Implementation Tasks**

You must:

- Explain external fragmentation using a diagram of the contiguous allocation
- Translate logical addresses to physical addresses with a page table
- Calculate the size of the page table for a 32-bit and a 48-bit address space with 4 KB pages
- Calculate the effective memory-access time with and without a TLB

**Evaluation**

You must:

- Explain how paging removes external fragmentation and why internal fragmentation remains
- Evaluate page sizes of 1 KB, 4 KB and 2 MB
- Explain hierarchical (multi-level), hashed and inverted page tables
- Explain how protection and shared pages are provided with paging

**Experimental Investigation**

You must:

- Calculate the internal fragmentation for 100 processes of random sizes with each page size
- Calculate the effective access time for TLB hit ratios of 80%, 95% and 99%
- Find the page size of your Linux system (getconf PAGESIZE) and the page faults of a program (/usr/bin/time -v)
- Test addresses at the start and end of a page and beyond the last page

**Conclusion**

*What page size would you choose for the college server, and why?*

**Complete the Code**

Complete TODO 1 below and execute the program.

```
#include <stdio.h>

#define PAGE_SIZE 1024
#define PAGES 8

/* Page table of a process: frame number, or -1 if the page is not in memory. */
int page_table[PAGES] = {5, 2, -1, 7, 0, -1, 3, -1};

int main(void)
{
    unsigned logical[] = {0, 1500, 3100, 6200, 9000, 2049};
    for (int i = 0; i < 6; i++) {
        unsigned a = logical[i];
        // TODO 1: Split a into page number p = a / PAGE_SIZE and offset d = a %
        //  PAGE_SIZE. If p is outside the table print "invalid address"; if the frame
        //  is -1 print "page fault"; otherwise print the physical address frame *
        //  PAGE_SIZE + d

    }
    return 0;
}
```

**Expected output:**

```
   0 -> page 0 offset  0 : physical 5120
1500 -> page 1 offset 476 : physical 2524
3100 -> page 3 offset  28 : physical 7196
6200 -> page 6 offset  56 : physical 3128
9000 -> page 8 offset 808 : invalid address
2049 -> page 2 offset  1 : page fault
```

Change PAGE_SIZE to 2048 and recalculate the results by hand before running the program again.

**Alternative Concept and Complexity Analysis**

If the server used an inverted page table (one entry per physical frame) instead of a per-process multi-level page table, what would the solution be? Evaluate the space used (proportional to physical memory versus to the address space of each process) and the time to translate an address (a hash look-up versus a walk of several levels).

---

## Q3 – Segmentation — 6 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Student Programs with Separate Code, Stack and Data**

Students' programs have code that should be read-only and shared by everyone running the same compiler, a stack that grows, and data of varying size. Paging alone treats all of these the same.

**Implementation Tasks**

You must:

- Translate (segment, offset) addresses using a segment table with base and limit
- Show what happens when an offset exceeds the segment limit
- Draw how segmentation with paging (as in the Intel x86 architecture) translates an address
- Show the segments of a running program on Linux (/proc/<pid>/maps)

**Evaluation**

You must:

- Explain how segmentation matches the programmer's view of memory
- Explain how segments are protected and shared
- Evaluate the external fragmentation caused by segmentation
- Compare pure paging, pure segmentation and segmented paging

**Experimental Investigation**

You must:

- Translate 8 addresses by hand and verify them with a small program
- Trigger a segmentation fault and explain what the OS did
- Calculate the fragmentation after loading and removing 20 segments of random sizes
- Compare the size of the tables needed in each scheme

**Conclusion**

*Why do modern operating systems rely mainly on paging?*

**Alternative Concept and Complexity Analysis**

If the programs used pure paging instead of segmentation, what would the solution be? Evaluate the fragmentation (internal versus external), the size of the tables, the time to translate an address, and how easily code is shared and protected.

---

## Q4 – Virtual Memory and Demand Paging 8 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Running Large Medical-Imaging Software on a Laptop**

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM.

**Implementation Tasks**

You must:

- Explain the steps of handling a page fault
- Calculate the effective access time for page-fault rates of 1 in 1,000, 1 in 100,000 and 1 in 1,000,000 (memory 100 ns, fault service 8 ms)
- Explain copy-on-write and how it makes fork() fast
- Explain thrashing and the working-set model

**Evaluation**

You must:

- Explain how demand paging lets a 24 GB program run in 16 GB
- Evaluate the maximum acceptable page-fault rate for a 10% slow-down
- Explain how the number of frames per process should be decided (equal and proportional allocation)
- Explain how the OS detects and stops thrashing

**Experimental Investigation**

You must:

- Write a program that touches memory in sequential and random order and count its page faults
- Measure the run time as the memory used grows beyond the free RAM (use a virtual machine or a memory limit)
- Show copy-on-write with fork() by measuring memory before and after writing
- Plot the page faults against the number of frames for a reference string

**Conclusion**

*Is buying more RAM or improving the program's locality the better solution for the radiologist? Justify.*

**Alternative Concept and Complexity Analysis**

If the radiologist's program were run on a machine with 32 GB of RAM instead of relying on demand paging, what would the solution be? Evaluate the effective access time in both cases for the measured page-fault rate, and the cost of extra memory against the cost of the slower run.

---

## Q5 – Page-Replacement Techniques — 8 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Choosing a Policy for a Web Server**

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to remove, and a poor choice causes slow responses.

**Implementation Tasks**

You must:

- Implement FIFO, LRU and Optimal page replacement
- Count the page faults for a given reference string with 3 and 4 frames
- Demonstrate Belady's anomaly with FIFO
- Explain the clock (second-chance) algorithm as a practical approximation of LRU

**Evaluation**

You must:

- Explain why Optimal is the best possible but cannot be implemented
- Explain why LRU never suffers from Belady's anomaly (the stack property)
- Evaluate the hardware support LRU needs and how the clock algorithm avoids it
- Compare the algorithms for a workload with strong locality

**Experimental Investigation**

You must:

- Generate reference strings with strong locality and with random access, and count the faults of each algorithm for 3 to 10 frames
- Plot the faults against the number of frames for each algorithm
- Measure the running time of each algorithm for 100,000 references
- Verify your program's results with a hand calculation for one string

**Engineering Decision**

> *Which page-replacement policy would you recommend for the web server? Use your measurements.*

**Complete the Code**

Complete TODO 1 below to finish LRU, and execute the program.

```c
#include <stdio.h>

#define MAXF 8

int find(const int f[], int n, int page)
{
    for (int i = 0; i < n; i++) if (f[i] == page) return i;
    return -1;
}

int fifo(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, next = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) f[used++] = ref[i];
        else { f[next] = ref[i]; next = (next + 1) % frames; }
    }
    return faults;
}

int lru(const int ref[], int len, int frames)
{
    int f[MAXF], last[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        int k = find(f, used, ref[i]);
        if (k >= 0) { last[k] = i; continue; }
        faults++;
        // TODO 1: If a frame is free use it; otherwise replace the frame whose last use
        //  (last[]) is the oldest. Record last[] = i for the frame used

    }
    return faults;
}

int optimal(const int ref[], int len, int frames)
{
    int f[MAXF], used = 0, faults = 0;
    for (int i = 0; i < len; i++) {
        if (find(f, used, ref[i]) >= 0) continue;
        faults++;
        if (used < frames) { f[used++] = ref[i]; continue; }
        int victim = 0, farthest = -1;
        for (int j = 0; j < frames; j++) {
            int next = len;        /* never used again */
            for (int t = i + 1; t < len; t++) if (ref[t] == f[j]) { next = t; break; }
            if (next > farthest) { farthest = next; victim = j; }
        }
        f[victim] = ref[i];
    }
    return faults;
}

int main(void)
{
    int ref[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1};
    int len = 20;
    for (int fr = 3; fr <= 4; fr++)
        printf("%d frames: FIFO %d, LRU %d, Optimal %d\n", fr, fifo(ref, len, fr),
               lru(ref, len, fr), optimal(ref, len, fr));
    int b[] = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5};
    printf("Belady string, FIFO: 3 frames %d faults, 4 frames %d faults\n",
           fifo(b, 12, 3), fifo(b, 12, 4));
    return 0;
}
```

**Expected output:**

```
3 frames: FIFO 15, LRU 12, Optimal 9
4 frames: FIFO 10, LRU 8, Optimal 8
Belady string, FIFO: 3 frames 9 faults, 4 frames 10 faults
```

Explain why FIFO has more faults with 4 frames than with 3 for the second reference string, and why LRU cannot behave like this.

**Alternative Concept and Complexity Analysis**

If the clock (second-chance) algorithm were used instead of exact LRU, what would the solution be? Evaluate the time per reference (updating a time stamp or a list on every access versus setting one bit), the extra space per frame, and how close its page faults come to LRU in your simulation.

---

## Q6 – Disk Management — 4 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Storage for a Hospital Records Server**

The hospital records server must start reliably, survive the failure of a disk and have enough swap space for peak load.

**Implementation Tasks**

You must:

- Explain low-level formatting, partitioning and logical formatting
- Explain the boot block and the boot process
- Explain how bad blocks are handled
- Decide the size and location of the swap space

**Evaluation**

You must:

- Compare RAID 0, 1, 5 and 10 for reliability, capacity and speed
- Choose the RAID level for the records server
- Evaluate swap on a separate disk versus a swap file
- Explain how disk partitions are viewed on Linux (lsblk, fdisk -l)

**Experimental Investigation**

You must:

- Show the partitions and swap of your Linux system
- Calculate the usable capacity of 6 disks of 4 TB at each RAID level
- Calculate the probability of data loss for RAID 1 and RAID 5 given a disk failure rate
- Measure the read speed of a file on SSD and hard disk if both are available

**Conclusion**

*Which storage configuration would you recommend for the records server, and why?*

**Alternative Concept and Complexity Analysis**

If the records server used RAID 6 instead of RAID 5 (or RAID 10), what would the solution be? Evaluate the usable capacity of 6 disks of 4 TB, the time to write a block (the extra parity calculation), and the number of disk failures that can be survived.

---

## Q7 – Design a Memory-Management Plan from First Principles 10 Marks

**Bloom's Level:** K5 – Evaluate

**Problem Context – Memory Management for an Edge Device in a Smart Factory**

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor wants to add more tasks in the future.

You are not given the design. You must develop it from the requirements.

**Stage 1 – Interpret the Problem | 2 Marks**

- Identify the memory needs and timing constraints of each task
- Identify which tasks can tolerate page faults and which cannot
- State the requirements as measurable statements

**Stage 2 – Compare Candidate Mechanisms | 3 Marks**

- Compare contiguous allocation, paging, segmentation and demand paging for each task
- Decide how memory is locked for the real-time task (for example mlock())
- Present a comparison table with justification

**Stage 3 – Implement and Verify | 3 Marks**

- Write a program that locks the memory of a time-critical loop and measure its worst-case delay with and without locking
- Simulate the memory use of the three tasks with a page-replacement simulator
- Verify that the real-time task is never delayed

**Stage 4 – Defend the Design | 2 Marks**

- Justify your plan using your measurements
- Explain what changes if a fourth task of 500 MB is added
- Identify one remaining limitation

**Alternative Concept and Complexity Analysis**

If the edge device used a real-time operating system with no virtual memory instead of Linux with locked memory for the control task, what would the solution be? Evaluate the worst-case delay of the control task, the memory available to the machine-learning task, and the effort of adding new tasks.

---

## MARK DISTRIBUTION

| Question | Assessed Area | Bloom's Level | Marks |
|---|---|---|---|
| Q1 | Linking, loading and address binding | K5 – Evaluate | 6 |
| Q2 | Paging | K5 – Evaluate | 8 |
| Q3 | Segmentation | K5 – Evaluate | 6 |
| Q4 | Virtual memory and demand paging | K5 – Evaluate | 8 |
| Q5 | Page-replacement techniques | K5 – Evaluate | 8 |
| Q6 | Disk management | K5 – Evaluate | 4 |
| Q7 | Memory-management plan from first principles | K5 – Evaluate | 10 |
| | **TOTAL** | | **50** |

### Submission Requirements

**GitHub repository link (mandatory)** containing the executed code for every task, including every completed "Complete the Code" program, script and configuration file, with input files, output screenshots and a README.md.

- Your report containing the required evidence for Q1–Q7
- C source code for all programs
- Address-translation workings, graphs, measurements and command outputs

### Assessment Verification

Students may be asked to explain, execute or modify any part of their design, code, calculations or results during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

# Question 1: Linking, Loading and Address Binding (6 Marks)

## 1.1 Problem context

A bank's branch software is built from many modules and libraries. When a security fix is made to the encryption library, every branch currently receives a new executable of about 200 MB. The question asks the team to compare static and dynamic linking, to measure their behaviour, and to decide which method the bank should use. The alternative concept asks what changes if the encryption library is loaded at run time with dlopen() instead of being linked at start-up.

## 1.2 Address binding

A program is a set of symbolic references to code and data. Before it runs, each reference must be bound to a concrete address. This binding can happen at three points.

**Compile-time binding** assigns absolute addresses when the program is compiled. It is only possible if the load address is known in advance. The output is fixed to one location, so the program cannot be moved. Early operating systems and small embedded systems used this method.

**Load-time binding** leaves the addresses relative until the program is loaded. The compiler emits relocation entries, and the loader patches each entry with the real address when it maps the program. The program can run at any base address, but once loaded it cannot move without being reloaded.

**Execution-time binding** keeps addresses as logical values until the instruction is executed. The hardware (the memory-management unit, MMU) translates each logical address to a physical one through the page table. This allows the OS to move pages between frames, swap them out, and share them. Modern general-purpose systems use execution-time binding for all user memory.

Shared libraries sit between these cases. Their code is position-independent and the dynamic linker binds their symbols at load or first-call time, so the binding is partly load-time and partly execution-time.

## 1.3 Roles of the linker and the loader

The **linker** (for example `ld`, invoked by `gcc`) takes object files and libraries and produces one output. It resolves references between modules, assigns section addresses, and applies relocations. With static linking, it copies the code of every referenced library routine into the executable. With dynamic linking, it records the names of the required shared libraries in the executable's dynamic section and leaves the symbol resolution for run time.

The **loader** runs when the program is started. For a statically linked program, the kernel maps the executable and sets up the stack and heap. For a dynamically linked program, the kernel maps the executable and then transfers control to the dynamic linker, `ld-linux.so`.

That program maps each required shared library, applies the relocations, resolves the symbols through the Global Offset Table (GOT) and the Procedure Linkage Table (PLT), and finally jumps to `main`.

## 1.4 Implementation

### Build the same program twice

Assume the program `bank.c` calls an encryption routine `encrypt()` and a logging routine. The encryption code lives in `libenc.c`. Build the static version with:

```
gcc -static bank.c -L. -lenc -o bank_static
```

and the dynamic version with:

```
gcc bank.c -L. -lenc -Wl,-rpath,'$ORIGIN' -o bank_dyn
```

The `-Wl,-rpath,'$ORIGIN'` option tells the dynamic linker to look for the library in the same directory as the executable.

### Compare executable sizes

```
ls -l bank_static bank_dyn
size bank_static bank_dyn
```

The static binary contains the library code and is far larger. The dynamic binary contains only its own code plus the symbol tables. The difference is the size of the library that every static executable carries. In the bank's case, if every branch application is statically linked and the library is 200 MB, the bank ships 200 MB with each application.

### List the shared libraries

```
ldd bank_dyn
```

The output lists `libc.so.6`, `ld-linux-x86-64.so.2`, and the encryption library. The `ldd` command runs the dynamic linker in a special mode to print the dependency tree. It does not run the program, so it is safe to use on an untrusted binary only through `objdump -p` (`ldd` can execute code on some untrusted files).

### Load a library at run time

The program in Appendix A.1 uses `dlopen()` to load `libenc.so`, `dlsym()` to find the address of `encrypt`, calls it, and then calls `dlclose()`. The function pointer returned by `dlsym()` is the only link between the program and the library. The program must check the return values of both calls, because a missing library is detected at this point and not at start-up.

### Show the load addresses

Inside a running program the file `/proc/self/maps` lists every mapped region with its address range and permissions. Running the dynamic version and reading this file shows the executable's text segment at a low address and the shared libraries at high, randomised

addresses. The randomisation is address space layout randomisation (ASLR). Run the command below twice to see that the library base changes between runs:

```
./bank_dyn & sleep 0.1; cat /proc/$!/maps
```

## 1.5 Evaluation

The two methods can be compared on four criteria.

**Memory use.** A statically linked program maps its own copy of the library, so ten running copies use ten copies of the library's code pages. A dynamically linked program maps the same library file. The read-only code pages are backed by the same physical frames, so the library's code is held in memory once no matter how many processes use it. Only the writable data pages are private to each process.

**Start-up time.** A static program needs no symbol resolution at start-up, so it starts a little faster. Dynamic programs must map the libraries and resolve symbols. With lazy binding the cost is paid on the first call to each function. The difference is usually a few milliseconds, which matters for short-lived tools and rarely matters for a long-running server.

**Ease of updating.** A dynamic program can be fixed by replacing the shared file. A static program must be relinked and redeployed in full. This is the main reason the bank's current process is slow: every fix ships 200 MB to every branch.

**Risk.** A dynamic program fails at load time if its library is missing or incompatible (the error message is "error while loading shared libraries"). A static program does not have this failure mode. This is a real cost of dynamic linking and must be handled by deployment tooling.

**Position-independent code.** A shared library must be mapped at different addresses in different processes, because ASLR and the process layout differ. Position-independent code (PIC) is compiled with `-fPIC`. Code accesses global data through the Global Offset Table, and calls to other functions through the Procedure Linkage Table, so the code bytes do not contain absolute addresses. The same physical code pages can therefore be mapped at different virtual addresses in different processes. Without PIC, the library's code would need relocation for each process, and the sharing benefit would be lost.

## 1.6 Experimental investigation

**Start-up time.** Use a repeated timing tool:

```
perf stat -r 100 ./bank_static
perf stat -r 100 ./bank_dyn
```

Record the mean elapsed time for each. Expect the static version to be slightly faster.

**Memory use.** Use:

```
/usr/bin/time -v ./bank_static
/usr/bin/time -v ./bank_dyn
```

Record Maximum resident set size. The static and dynamic versions should show similar resident sizes for a single process, because both map the library. The difference appears when many processes run.

**Replace the library without recompiling.** Build `libenc.so` from a version that returns `x*31+7`. Run `LD_LIBRARY_PATH=. ./bank_dyn`. Replace the source with `x*31+9`, rebuild only the shared file, and run again. The output changes while the executable is unchanged. Confirm with `md5sum bank_dyn` before and after. Then run the static binary and show that its output does not change until it is relinked.

**Two copies sharing pages.** Start two instances of `bank_dyn` and inspect the libc mapping in each:

```
./bank_dyn & P1=$!
./bank_dyn & P2=$!
grep -A3 libc /proc/$P1/smaps | head -20
grep -A3 libc /proc/$P2/smaps | head -20
```

The `Shared_Clean` field shows the code pages that are shared. The `Private_Dirty` field shows the writable pages that are private. Record the values and explain the difference.

**Missing library.** Rename `libenc.so` and run `bank_dyn`. Record the error. Then run `bank_static` and show that it still runs. Explain that the failure happens in the dynamic linker before `main` is entered.

## 1.7 Alternative concept: dlopen only when needed

Consider loading the encryption library with `dlopen()` only when the first encryption request arrives, instead of linking it at start-up.

**Start-up time** improves, because the library is not mapped, relocated, or symbol-resolved until it is used. A branch application that never encrypts (for example a read-only reporting tool) pays no cost at all.

**Memory use** is lower for those runs, because the library's pages are not in memory. For runs that do encrypt, memory is the same as the normal dynamic case.

**Error handling** moves from start-up to run time. `dlopen()` and `dlsym()` return NULL on failure, and the program must check the result and decide what to do. For a banking system the decision matters: the request must either fail cleanly with a useful message, or the program must refuse to start until the library is verified. The bank should not discover a missing library in the middle of a transaction. A sound design loads the library at a defined point, verifies its version and checksum, and reports failure before accepting customer work.

**Security** also changes. A library loaded on demand from a directory the process can write to is a risk. The bank should load it only from a root-owned directory with a checked hash.

### 1.8 Conclusion

Dynamic linking is the right choice for the bank. It keeps the applications small, shares the library's code pages between processes, and allows the security fix to be deployed by replacing one shared file. The cost is a start-up dependency on the library, which deployment tooling can control. Static linking is preferable only where a deployment must be self-contained and must never fail because of a missing file, for example on an isolated kiosk. For the branch software, the recommended design is dynamic linking with a version check at start-up and optional on-demand loading for rarely used modules.

## Question 2: Paging (8 Marks)

### 2.1 Problem context

A college server runs hundreds of student programs of different sizes. Contiguous memory allocation creates many small holes that cannot be used. The question asks the team to explain fragmentation, translate addresses, size page tables, calculate effective access time with and without a TLB, and choose a page size.

### 2.2 External fragmentation

Under contiguous allocation each process occupies one unbroken block of physical memory. When a process ends, its block becomes a hole. Over time the memory contains many holes of different sizes. External fragmentation is the condition in which the total free memory is large enough for a new process, but no single hole is large enough.

Consider 100 KB of memory with three processes P1 (30 KB), P2 (20 KB) and P3 (40 KB) loaded in that order, then P2 terminates. Free memory is 20 KB in the middle and 10 KB at the end. A new 25 KB process cannot be placed, although 30 KB is free. A diagram of the memory layout makes this clear:

```
 0 KB   +-----------+
        |    P1     |  30 KB
30 KB   +-----------+
        |   hole    |  20 KB   (P2 removed)
50 KB   +-----------+
        |    P3     |  40 KB
90 KB   +-----------+
        |   hole    |  10 KB
100 KB  +-----------+
```

Compaction can remove the holes but costs time proportional to the memory moved. Paging avoids the problem by making every unit of allocation the same size.

### 2.3 Address translation with a page table

Paging divides the logical address space into fixed-size pages and the physical memory into frames of the same size. A logical address is split into a page number and an offset:

- page number p = floor(logical address / page size)
- offset d = logical address mod page size

The page table maps each page number to a frame number f. The physical address is f × page size + d.

**Worked example.** Page size 4 KB (4096 bytes). Logical address 8200. Then p = floor(8200 / 4096) = 2 and d = 8200 − 8192 = 8. If the page table entry for page 2 holds frame 7, the physical address is 7 × 4096 + 8 = 28672 + 8 = **28680**.

The same translation is performed by a small C program (Appendix A.2), which is also the basis of the experiment that verifies it.

## 2.4 Page table size

For a single-level page table, the number of entries is the number of pages in the address space, which is 2^(address bits) / page size.

**32-bit address space, 4 KB pages.** 2^32 / 2^12 = 2^20 = 1,048,576 entries. If each entry is 4 bytes, the table is 4 MB per process. Hundreds of processes would need gigabytes of tables.

**48-bit address space, 4 KB pages.** 2^48 / 2^12 = 2^36 = 68,719,476,736 entries. With 8-byte entries the table is 2^39 bytes = 512 GB per process. No system holds this in memory for every process, which is why the 48-bit spaces of current processors use multi-level or hashed tables.

These values assume a flat table. Multi-level tables allocate only the parts of the table that map used regions, so a typical process needs far less.

## 2.5 Effective memory-access time

Assume a memory access takes 100 ns and a TLB lookup takes 20 ns.

**Without a TLB.** Each logical reference needs the page table in memory (100 ns) and then the data (100 ns). The effective access time is 200 ns.

**With a TLB.** The TLB holds recently used page-table entries. On a hit, the translation takes 20 ns and the data access takes 100 ns, for 120 ns. On a miss, the TLB lookup (20 ns) is followed by a page-table access (100 ns) and the data access (100 ns), for 220 ns. With hit ratio h,

EAT = h × 120 + (1 − h) × 220

| Hit ratio | Calculation | EAT |
|---|---|---|
| 80% | 0.80 × 120 + 0.20 × 220 | **140 ns** |
| 95% | 0.95 × 120 + 0.05 × 220 | **125 ns** |
| 99% | 0.99 × 120 + 0.01 × 220 | **121 ns** |

The TLB reduces the overhead of paging from 200 ns to 121 ns at a 99% hit ratio. A hit ratio of 80% still gives a 30% gain over no TLB, and the gain grows quickly above 95%, which is why processors invest heavily in TLB design.

## 2.6 Internal fragmentation and the choice of page size

Internal fragmentation is the memory wasted inside the last page of each process. If the process size is random, the last page is on average half full, so the expected waste per process is P/2 for page size P.

For 100 processes:

| Page size | Expected waste per process | Total for 100 processes |
|---|---|---|
| 1 KB | 0.5 KB | about 50 KB |
| 4 KB | 2 KB | about 200 KB |
| 2 MB | 1 MB | about 100 MB |

Measure the actual values with a script that draws 100 random sizes (for example uniform between 1 KB and 500 KB), computes `page_size - (size mod page_size)` for each size where the remainder is nonzero, and sums the results. The totals will differ from the expectation by sampling error, but the ranking will hold.

Smaller pages reduce internal fragmentation but increase the page-table size, the number of TLB entries needed, and the number of disk transfers. Larger pages do the opposite. The choice of 4 KB balances these costs.

## 2.7 Hierarchical, hashed and inverted page tables

**Hierarchical (multi-level) tables** split the logical page number into several indices. A two-level table uses the first index to find a second-level table and the second index to find the frame. Only the second-level tables for used regions exist, so sparse address spaces need little memory. The cost is one extra memory access on a TLB miss for each level.

**Hashed tables** hash the virtual page number to find a bucket that holds a chain of (page, frame) entries. They suit sparse 64-bit spaces because the table size depends on the number of mapped pages, not the address range.

**Inverted tables** keep one entry per physical frame. Each entry records which process and which page occupies the frame. The table size is fixed by the amount of physical memory, not by the number of processes. Lookup requires a search, usually replaced by a hash table, and sharing a frame between processes needs extra bookkeeping.

## 2.8 Protection and shared pages

Each page-table entry carries protection bits: valid, read, write and execute. The MMU checks them on every access. A write to a read-only page raises a protection fault, and the kernel decides whether to deliver a signal.

Shared pages are two or more page-table entries that point to the same frame. Library code is the standard case: each process maps the same read-only frames. Writable shared memory is also possible, and the protection bits decide who may write. The kernel does not need to copy shared read-only pages, which is the main memory saving.

### 2.9 Experimental investigation

**Linux page size.** Run:

```
getconf PAGESIZE
```

On typical x86-64 Linux the output is 4096. Record the value and state that it matches the 4 KB assumption.

**Page faults.** Run:

```
/usr/bin/time -v ./student_prog
```

Record the "Major (requiring I/O) page faults" and "Minor (reclaiming a frame) page faults" lines. A program that touches a large array for the first time produces many minor faults, one per page, because the kernel allocates each frame on first touch.

**Boundary tests.** Use a program that allocates a mapped region and then reads at offsets 0, 4095, 4096, and beyond the last page. Offsets inside the mapping succeed. The first access past the mapped region raises SIGSEGV. Appendix A.2 gives the translation program that performs the same boundary checks on the page table of the example.

**Internal fragmentation.** Use the script described in section 2.6 and record the totals for 1 KB, 4 KB and 2 MB.

### 2.10 Conclusion

Paging removes external fragmentation because any free frame can hold any page. Internal fragmentation remains, but it is bounded by one page per process. For the college server, 4 KB pages are the right choice. They keep the expected waste small compared with the process sizes, keep page tables a manageable size, match the Linux default, and match the granularity of hardware features such as TLB entries. A 2 MB page could be used for a few large, long-running programs, but it would waste memory across hundreds of small student programs.

## Question 3: Segmentation (6 Marks)

### 3.1 Problem context

Student programs have code that should be read-only and shared, a stack that grows, and data of varying sizes. The question asks the team to translate segmented addresses, to show what happens on a limit violation, to combine segmentation with paging, to inspect a running program, and to compare the three schemes.

## 3.2 Segmentation and the programmer's view

Segmentation divides a program into logical units: code, global data, heap, stack, and each library. The programmer refers to an item by a segment number and an offset within the segment. The mapping from segment to physical memory is kept in a segment table.

Each segment table entry holds a base (the physical start address) and a limit (the length). A logical address (s, d) is valid if d < limit[s]. The physical address is base[s] + d.

## 3.3 Translation with a segment table

Use the following segment table:

| Segment | Name | Base | Limit |
|---|---|---|---|
| 0 | Code | 1400 | 1000 |
| 1 | Data | 4300 | 1100 |
| 2 | Stack | 6300 | 400 |

The hand translations are:

- (0, 53): offset 53 < 1000, so the physical address is 1400 + 53 = **1453**.
- (2, 399): offset 399 < 400, valid, physical address 6300 + 399 = **6699** (the last valid byte).
- (2, 400): offset 400 is not less than 400, so the access is a **trap**.
- (1, 1099): offset 1099 < 1100, valid, physical address 4300 + 1099 = **5399**.
- (1, 1100): **trap**.
- (0, 999): valid, physical address 2399.
- (0, 1000): **trap**.
- (2, 0): valid, physical address 6300.

These eight translations are the ones repeated by the verification program in Appendix A.3.

## 3.4 What happens when the offset exceeds the limit

When the MMU compares the offset with the limit and finds it is not smaller, it raises a trap, a hardware exception. The trap transfers control to the kernel. The kernel looks at the faulting address and checks whether it belongs to a valid region of the process. If not, the kernel sends SIGSEGV, and the default action terminates the process and writes a core dump if enabled. The message "Segmentation fault (core dumped)" comes from the shell reporting this signal.

The same mechanism protects the program from accidental overruns of an array into another segment. The overrun is caught at the segment boundary, not after it has corrupted other data.

## 3.5 Segmentation with paging

Pure segmentation has external fragmentation, because segments are variable in size. Combining segmentation with paging keeps the logical view and removes the fragmentation. Each segment has its own page table. The segment number selects the page table, the offset is split into a page number and an offset within the page, and the page table gives the frame.

The translation for (s, d) is:

1. Look up segment s in the segment table to find its page table and limit.
2. Check d < limit[s], otherwise trap.
3. Split d into page number p and page offset o.
4. Look up frame f in the page table for segment s at index p.
5. Physical address = f × page size + o.

Intel's 32-bit x86 combined both schemes in this way.

## 3.6 Showing segments on Linux

For a running process with PID `<pid>`, run:

```
cat /proc/<pid>/maps
```

Each line gives an address range, permissions and the backing object. The lines are segments in the logical sense: the executable text (r-xp), the data and BSS (rw-p), the heap (marked [heap]), the stack (marked [stack]), and each shared library. The permission field shows the protection that the segment table would hold, and the lines marked `p` (private) are copy-on-write.

Record the output for a simple program, and identify each line with its segment type.

## 3.7 Evaluation

**Match to the programmer's view.** Segments correspond to code, data, stack and heap. A programmer thinks in these units, so the segment is the natural unit for protection and sharing.

**Protection and sharing.** Each segment has its own limit and access rights. The code segment is read-only and can be shared between processes that run the same program. Data and stack segments are private. Sharing a segment needs only one segment-table entry in each process pointing at the same physical base.

**External fragmentation.** Segments have different sizes. Loading and removing segments leaves holes just as in contiguous allocation. The student server would see the same problem as in Q2 without a paging layer. The remedy is compaction, which is expensive, or paging.

**Comparison.**

| Property | Pure paging | Pure segmentation | Segmented paging |
|---|---|---|---|
| External fragmentation | None | Yes | None |
| Internal fragmentation | Up to one page per unit | None | Up to one page per segment |
| Matches programmer's view | No | Yes | Yes |
| Table size | One table per process | One small table per process | One table per segment |
| Translation steps | One lookup | One lookup | Two lookups |
| Sharing and protection | By page | By segment | By segment, then page |

## 3.8 Experimental investigation

**Translate eight addresses by hand and verify.** The hand translations in section 3.3 are reproduced by the program in Appendix A.3. Run it and compare the output.

**Trigger a segmentation fault.** Write to a read-only page, or dereference NULL. A simple test:

```
int main(void) { int *p = NULL; *p = 1; return 0; }
```

Run it and record the signal. Explain that the MMU found no valid mapping for address 0, the kernel had no region there, and it delivered SIGSEGV.

**Fragmentation after 20 segments.** Allocate 20 segments of random sizes into a 10,000-unit memory, free 10 of them at random, and compute total free memory minus the largest free hole. Record the value. This is the fragmentation that a paged system would avoid.

**Table sizes.** For 100 processes with 4 segments each, compare the total number of table entries needed by pure segmentation (400 entries), pure paging with 4 KB pages (one table per process of 2^20 entries in the 32-bit case), and segmented paging (400 segment entries plus the page tables for the segments in use).

## 3.9 Conclusion

Modern operating systems rely mainly on paging. Paging removes external fragmentation, allocates memory in fixed frames that the kernel can hand out freely, and keeps the table format simple. Segmentation's benefits, the logical view, protection and sharing, are kept through per-region protection bits in the page table and through separate virtual memory areas, as the Linux maps in section 3.6 show. Hardware segmentation support is now minimal, so the segment idea survives mainly as an organisation of virtual regions.

# Question 4: Virtual Memory and Demand Paging (8 Marks)

## 4.1 Problem context

A radiologist needs to run an image-processing program that uses 24 GB of memory on a laptop with 16 GB of RAM. The question asks the team to explain page-fault handling, calculate effective access time for three fault rates, explain copy-on-write, explain thrashing, and recommend between buying memory and improving locality.

## 4.2 Demand paging

With demand paging, a page is loaded into memory only when a process first references it. The page table entry starts as invalid for pages not yet in memory. The program can therefore be larger than physical memory, as long as the pages it actively uses fit. A 24 GB program on 16 GB of RAM runs if its working set at any time is smaller than the frames available to it.

## 4.3 Steps in handling a page fault

1. The process references a virtual address. The MMU finds the page-table entry invalid and traps to the kernel.
2. The kernel checks that the address belongs to a valid region of the process. If not, it sends SIGSEGV and stops.
3. The kernel finds a free frame. If no frame is free, it runs page replacement, chooses a victim frame, and writes it to swap if it is dirty.
4. The kernel schedules a disk read that brings the requested page into the frame.
5. The process is blocked, and the CPU runs another process while the read completes.
6. When the read completes, the kernel updates the page table entry to valid with the frame number.
7. The faulting instruction is restarted, and the access now succeeds.

The dominant cost is step 4: a disk read takes milliseconds, while a memory access takes nanoseconds.

## 4.4 Effective access time

Let p be the probability of a page fault per memory reference. The memory access time is 100 ns and the fault service time is 8 ms = 8,000,000 ns. The effective access time is

EAT = (1 − p) × 100 + p × 8,000,000

**p = 1/1,000.** EAT = 0.999 × 100 + 0.001 × 8,000,000 = 99.9 + 8,000 = **8,099.9 ns, about 8.1 µs**. Paging overhead is about 80 times the memory access time.

**p = 1/100,000.** EAT = 0.99999 × 100 + 0.00001 × 8,000,000 = 99.999 + 80 = **179.999 ns, about 180 ns**.

**p = 1/1,000,000.** EAT = 0.999999 × 100 + 0.000001 × 8,000,000 = 99.9999 + 8 = **107.9999 ns, about 108 ns**.

The lesson is that the fault rate must be extremely low. Even one fault in a thousand accesses increases the access time eighty-fold.

## 4.5 Maximum fault rate for a 10% slowdown

A 10% slowdown means the EAT is at most 110 ns. Solve

110 = (1 − p) × 100 + p × 8,000,000

which gives 110 = 100 + p × (8,000,000 − 100), so p = 10 / 7,999,900 ≈ **1.25 × 10^-6**. This is roughly one fault per 800,000 memory references. Any workload that faults more often than this is slowed by more than 10%.

## 4.6 Copy-on-write and fork()

The `fork()` system call creates a child process with a copy of the parent's address space. Copying every page would be slow and wasteful, because the child usually calls `exec()` at once and discards the copy. Copy-on-write (COW) avoids the copy.

After `fork()`, the kernel copies the page tables, not the pages. Both page tables point to the same frames, and the shared pages are marked read-only. When either process writes to a shared page, the MMU raises a protection fault. The kernel then allocates a new frame, copies the page into it, points the writing process's entry at the new frame with write permission, and resumes the write. Only the pages that are written get copied.

**Measurement procedure.** Write a program that allocates 1 GB, touches every page so that it is resident, forks, and then reads `VmRSS` from `/proc/self/status` in the child before and after writing to a portion of the memory. The resident set of the child rises only by the pages written. Record the values for a 10% write and a 100% write, and explain the difference.

## 4.7 Thrashing and the working set

The working set of a process at time t is the set of pages referenced in the window of the last Δ references. If the total of all working sets exceeds the number of frames, the system cannot keep every active set in memory. Each process evicts pages that another process needs soon. The result is a high page-fault rate, the CPU sits idle waiting for disk, and throughput falls sharply. This collapse is thrashing.

The working-set model prevents thrashing by giving each process enough frames to hold its working set. The OS may suspend processes when the total demand exceeds memory, so that the rest run well.

## 4.8 Detecting and stopping thrashing

The OS can detect thrashing by monitoring the page-fault frequency of each process. If a process faults more than an upper threshold, it is given more frames. If it faults less than a lower threshold, frames are taken from it. When no free frames remain and the upper threshold is still exceeded, a process is suspended and its frames are released. This is page-fault frequency control.

A simpler sign is CPU utilisation: when utilisation falls while the paging device is busy, the system is thrashing. The OS responds by lowering the multiprogramming level, that is, by suspending processes.

## 4.9 Frames per process

The number of frames a process should receive is the size of its working set, in pages. The working set can be estimated by sampling reference bits over an interval, or from measured fault rates. A process with a small working set can run in a few frames. A process with a large working set needs most of the memory or runs slowly regardless of allocation.

## 4.10 Experimental investigation

**Sequential and random access.** Write a program that allocates an array of N pages, touches every page sequentially for several passes, and then touches them in a random order. Use `getrusage` before and after each phase to count minor and major faults. Random access produces far more faults once the array is larger than the available frames.

**Runtime beyond free RAM.** Run the program with increasing array sizes, from well below free RAM to beyond it. Plot runtime against size. The curve is nearly linear while the data fits, and then rises steeply once paging begins.

**Copy-on-write.** As described in section 4.6.

**Faults against frames.** Use a fixed reference string and the simulator in Appendix A.4 with 3 to 10 frames, and plot the fault count. For a string with locality the curve drops quickly and then flattens.

## 4.11 Conclusion

The radiologist’s program has a working set that determines whether it runs well. The first step should be to improve locality: process the image in tiles that fit in memory, reuse buffers, and avoid repeated passes over the whole data set. These changes can cut faults by orders of magnitude at no hardware cost. Buying more RAM is the right answer only if the working set itself exceeds 16 GB. Measuring the working set before buying memory avoids spending money on a problem the program can fix itself.

## Question 5: Page-Replacement Techniques (8 Marks)

### 5.1 Problem context

A web server keeps frequently used pages of data in memory. When memory is full, the OS must choose which page to evict. The question asks the team to implement FIFO, LRU and Optimal, to count faults with 3 and 4 frames, to demonstrate Belady's anomaly, to explain the clock algorithm, and to recommend a policy.

### 5.2 Algorithms

**FIFO** evicts the page that has been in memory longest. It needs only a queue.

**Optimal (OPT)** evicts the page whose next reference is farthest in the future, or never. It produces the fewest possible faults for any reference string, but it needs knowledge of future references, so it cannot be implemented in an operating system. Its value is as a benchmark.

**LRU** evicts the page that has gone unused for the longest time. It approximates OPT using the past as a predictor of the future. Exact LRU requires updating a record on every reference.

**Clock (second chance)** keeps the frames in a circular list with one reference bit each. The hand advances. If the page at the hand has its bit set, the bit is cleared and the hand moves on. If the bit is clear, the page is evicted. The reference bit is set by hardware on every access.

### 5.3 Results for the Belady string

The reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 is the classic string that shows Belady's anomaly. A simulator (Appendix A.4) and hand traces (Appendix B) give the following fault counts.

| Algorithm | 3 frames | 4 frames |
|---|---|---|
| FIFO | 9 | 10 |
| LRU | 10 | 8 |
| Optimal | 7 | 6 |

FIFO faults increase when the number of frames increases from 3 to 4. This is Belady's anomaly. LRU and Optimal decrease, as expected.

**Course reference string.** The expected results for the course's reference string (FIFO 15, LRU 12, Optimal 9 at 3 frames; FIFO 10, LRU 8, Optimal 8 at 4 frames) belong to a different string from the Belady test. Run the simulator on the course string to verify those figures. If the string is supplied, the team should enter it into the simulator and record the results in Table 5.1.

## 5.4 Why Optimal is the best possible but cannot be implemented

Optimal's decision depends on the future reference sequence. A kernel cannot know which page a process will touch next, because the process's future depends on inputs not yet seen. Optimal therefore cannot be implemented as a policy. Its value is that it gives a lower bound: no real algorithm can beat it, so the gap between a real algorithm and Optimal measures how much room for improvement exists.

## 5.5 Why FIFO has more faults with 4 frames

Belady's anomaly occurs because FIFO ignores how recently or how often a page has been used. With more frames it keeps pages longer, but the particular pages it keeps are not necessarily the ones that will be needed. In the Belady string, the 4-frame run evicts page 1 when page 5 arrives, and page 1 is needed immediately after. The 3-frame run had evicted a different page at that point, which happened to be the one needed later. The extra frame changes the eviction order, and the new order causes more faults.

## 5.6 Why LRU cannot show Belady's anomaly

LRU is a stack algorithm. For any reference string and any number of frames n, the set of pages in memory with n frames is a subset of the set with n + 1 frames. The proof is by induction on the reference string: the n most recently used distinct pages are always the top of the stack, and the top n + 1 include the top n. Adding a frame therefore can only keep pages that would otherwise have been evicted, so faults cannot increase.

FIFO does not have this property. Its eviction order is based on arrival time, which does not change when frames are added.

## 5.7 Hardware support for LRU

Exact LRU needs to know, at eviction time, the least recently used frame. Three ways of doing this are:

- a counter for each page-table entry, copied into a time-of-use field on every reference, with the minimum found by search at eviction;
- a stack of page numbers, with the referenced page moved to the top on every reference, which requires a list update per access;
- a matrix of bits, where setting row and column bits marks recency.

Each of these needs work on every memory reference or a search at every fault. Hardware support could store a counter in each entry and update it on every access, but this cost is high in silicon and cycles. Real systems therefore use approximations such as clock, which needs only one reference bit per frame, set by hardware.

## 5.8 Clock algorithm

The clock algorithm arranges the frames in a circle with a hand pointing at one frame. On a fault:

1. Examine the frame at the hand.
2. If its reference bit is 1, clear it, advance the hand, and repeat.
3. If its reference bit is 0, replace the page in that frame, load the new page, set its reference bit, and advance the hand.

A page that was referenced recently has its bit set and gets a second chance. The hand sweeps over pages that have not been used since the last sweep and evicts them. The extended version uses two bits (reference and dirty) so that clean pages are preferred for eviction, which avoids writing to disk.

## 5.9 Experimental investigation

**Reference strings.** Generate two kinds of string with Python. A strong-locality string has 80% of references drawn from a set of 10 pages and 20% from the other pages. A random string draws uniformly from 50 pages. Each string has 100,000 references.

**Fault counts.** Run each algorithm with 3 to 10 frames on each string. Record the results in a table and plot faults against frames. The Belady string shows the anomaly for FIFO. The strong-locality string shows LRU and clock close to Optimal and FIFO well above them.

**Running time.** Time each algorithm on 100,000 references with `time.perf_counter()`. The expected order is FIFO fastest, then clock, then LRU, with Optimal slowest because it scans the future at each fault.

**Hand verification.** Choose a short string of 12 references and verify each algorithm's fault count by hand. Appendix B gives the traces for the Belady string as a model.

**Complete the code (TODO 1).** The LRU routine must (a) on a hit, update the last-use time of the page to the current index; (b) on a fault with full frames, evict the frame whose last-use time is smallest; (c) load the new page and record the current index; (d) count each fault. The simulator in Appendix A.4 is a complete reference implementation to check the C version against.

## 5.10 Engineering decision

Recommend the clock algorithm for the web server. The workload of a web-page cache has strong locality: a few popular pages are requested most of the time. Clock approximates LRU closely on such workloads, needs one bit per frame, and has constant time per reference with no per-access bookkeeping. Choose the number of frames after measuring the fault curve; the knee of the curve shows the working-set size.

## 5.11 Alternative concept: clock instead of exact LRU

Time per reference: exact LRU requires an update of recency information on each access, which is expensive when accesses are frequent. Clock does no work on a hit beyond setting the reference bit, which hardware does automatically. Extra space: exact LRU needs a counter or list per frame; clock needs one bit per frame and a hand pointer. Page faults: clock's fault count is usually close to LRU's and well below FIFO's on workloads with

locality, but it can be slightly higher than LRU on some strings, because it does not know the exact order of recent use.

### 5.12 Conclusion

For the web server, clock gives the best trade of fault count against cost. The recommendation depends on the measured curves, which the team should record and attach.

## Question 6: Disk Management (4 Marks)

### 6.1 Problem context

A hospital records server must start reliably, survive disk failure, and have enough swap for peak load. The question asks the team to explain disk preparation and booting, to describe bad-block handling, to size swap, to compare RAID levels, to decide between swap partitions and swap files, and to recommend a configuration.

### 6.2 Low-level formatting, partitioning and logical formatting

**Low-level formatting** is performed by the disk manufacturer or a utility. It divides the disk surface into sectors, writes sector headers and checksums, and marks defective sectors. Modern drives do this at the factory, and the operating system does not normally repeat it.

**Partitioning** divides the disk into regions. The partition table records the start, end and type of each region. MBR stores four primary partitions in the first sector, and GPT stores a larger table with checksums, supporting disks over 2 TB.

**Logical formatting** creates a file system in a partition: superblock, inode or allocation tables, and an empty root directory. It is what `mkfs` does.

### 6.3 Boot block and the boot process

On a BIOS system the firmware reads the first sector of the disk (the master boot record), finds the active partition, reads its boot sector, and runs the boot loader found there. On a UEFI system the firmware reads the EFI system partition, a FAT partition listed in the GPT, and runs the boot loader file in it.

The boot loader (for example GRUB) loads the kernel and the initial RAM disk into memory, then passes control to the kernel. The kernel mounts the root file system and starts the init process. A failure in any stage stops the boot, so the boot partition must be on a reliable disk and must be backed up.

### 6.4 Bad blocks

A bad block is a sector that cannot be read or written reliably. Two methods handle it.

**Sector sparing.** The disk controller maps a bad sector to a reserved spare sector. The operating system sees a good address space, and the substitution is invisible. Most modern drives do this internally.

**Bad-block list.** The operating system maintains a list of bad blocks in the file system, marks them as allocated so they are never used, and may re-test them later. The `badblocks` tool finds them, and `e2fsck -l` records them.

For the records server, monitor SMART attributes (reallocated sector count, pending sector count) and replace a disk when these grow, rather than waiting for failure.

## 6.5 Size and location of swap

Swap holds pages that cannot stay in RAM. Its size depends on the workload. A common rule is to size swap to match RAM for hibernation support, and to provide enough extra swap for peak demand. For a 16 GB records server, 8 to 16 GB of swap is reasonable, with monitoring to show whether it is used at all in normal operation. Heavy swapping on a records server is itself a warning sign: it means the memory is undersized for the load.

Swap on a separate disk keeps paging I/O away from the data disks, so that a paging storm does not slow record access. A swap file is simpler to resize and needs no partition, but it lives on a file system, adds a small overhead, and may be fragmented. On a server with a dedicated spare disk, a swap partition on that disk is the better choice. On a single-disk server, a swap file is acceptable.

## 6.6 RAID comparison

Six 4 TB disks are assumed.

| Level | Usable capacity | Failures survived | Read speed | Write speed |
|---|---|---|---|---|
| RAID 0 (striping) | 24 TB | 0 | Very fast | Very fast |
| RAID 1 (mirror, 3 pairs) | 12 TB | 1 per pair | Fast | Normal |
| RAID 5 (single parity) | 20 TB | 1 | Fast | Slow (parity updates) |
| RAID 6 (double parity) | 16 TB | 2 | Fast | Slower than RAID 5 |
| RAID 10 (mirror of stripes) | 12 TB | 1 per mirror pair | Very fast | Fast |

Capacity: RAID 0 uses all six disks, RAID 1 halves them, RAID 5 loses one disk to parity (5 × 4 = 20 TB), RAID 6 loses two (4 × 4 = 16 TB), and RAID 10 halves them.

## 6.7 Probability of data loss

Assume an annual failure rate of 2% per disk. The per-hour failure rate is λ = 0.02 / 8,760 ≈ 2.3 × 10^-6. Assume a rebuild takes 24 hours.

**RAID 1.** A failed disk is rebuilt from its partner. Data is lost if the partner fails during the rebuild. The probability is about λ × 24 ≈ 5.5 × 10^-5 for each disk failure.

**RAID 5 with six disks.** Data is lost if any of the other five disks fails during rebuild. The probability is about 5 × λ × 24 ≈ 2.7 × 10^-4 for each disk failure.

RAID 5 is about five times more exposed in this model, and the exposure grows with the number of disks and the length of the rebuild. The estimates are approximate because they assume independent failures; real disks in the same enclosure fail together more often, so the true risk is higher. The team should recalculate with the vendor's failure rate and the measured rebuild time.

## 6.8 Recommendation

Recommend RAID 10. It gives the best speed and survives one failure per mirror pair, and its rebuild reads only the surviving partner of the failed disk, so the exposure window is short. The capacity cost is half the raw space. For a records server whose data is also backed up off-site, this is the right balance. RAID 5 is cheaper per usable terabyte but carries the larger rebuild risk shown above. RAID 6 is the choice if capacity matters more than write speed and two-failure tolerance is required.

RAID is not a backup. The recommendation must include regular backups and tested restores.

## 6.9 Commands and measurements

- `lsblk` lists block devices, sizes and mount points.
- `sudo fdisk -l` lists partition tables.
- `swapon --show` and `free -h` show swap devices and use.
- For the read-speed experiment, run `dd if=<file> of=/dev/null bs=1M` on a file on each device, after clearing the cache with `echo 3 | sudo tee /proc/sys/vm/drop_caches`, and record the reported throughput.

## 6.10 Alternative concept: RAID 6 instead of RAID 5 or RAID 10

RAID 6 survives any two disk failures, which neither RAID 5 nor a single failure per RAID 10 pair guarantees. Usable capacity with six 4 TB disks is 16 TB, between RAID 5 (20 TB) and RAID 10 (12 TB). Write time is higher than RAID 5 because each write updates two parity blocks, and the read-modify-write penalty is significant. RAID 6 is the right choice when losing two disks during a rebuild is a serious risk, which is the case for large, slow-rebuilding arrays.

### 6.11 Conclusion

For the records server, RAID 10 on a dedicated controller, a swap partition on a separate disk, SMART monitoring, and off-site backups give reliable start-up, tolerance of disk failure, and enough swap for peaks.

## Question 7: Designing a Memory-Management Plan (10 Marks)

### 7.1 Problem context

A smart factory installs edge devices with 2 GB of RAM and no swap disk. Each device runs a real-time control task that must never be delayed by a page fault, a machine-learning task that needs 1.5 GB, and a logging task. The vendor plans to add tasks later. This question asks for a design developed from first principles, with measurements that justify it.

### 7.2 Stage 1: Interpret the problem

**Memory needs.** The control task needs a small fixed footprint, assumed here to be under 50 MB for code, data and stack. The machine-learning task needs 1.5 GB. The logging task needs a bounded buffer, assumed to be 100 MB. The kernel and system services need about 200 MB. The total is about 1.85 GB, leaving about 150 MB of headroom. The margin is small and it is the key constraint.

**Timing constraints.** The control task has a deadline, and a single page fault costs about 8 ms in the worst case with a disk, which is far beyond any control period. The control task cannot tolerate faults at all. The machine-learning task has a throughput goal rather than a deadline, so it can tolerate faults if they are rare. The logging task has no timing constraint.

**Tasks that tolerate faults.** The logging task and, within limits, the machine-learning task. With no swap, a fault on a page that is not resident cannot be served from disk for anonymous memory. Faults are therefore only minor (for example file-backed pages from a read-only binary) or the process is killed. This changes the question: with no swap, the design must avoid major faults altogether, which means keeping all anonymous memory resident.

**Measurable requirements.**

- R1: The control task's worst loop iteration time, measured with `clock_gettime`, is within its deadline, and its minor and major fault counts from `getrusage` stay at zero after warm-up.
- R2: The machine-learning task reaches a resident set of 1.5 GB without swap and without being killed by the out-of-memory killer.
- R3: The logging task's buffer is bounded at 100 MB.
- R4: Total committed memory stays below 1.9 GB, leaving 10% headroom for the kernel.

## 7.3 Stage 2: Compare candidate mechanisms

**Contiguous allocation.** Each task receives one region. This is simple and predictable, and the control task could be given one fixed region at boot. The disadvantage is external fragmentation as tasks start and stop, and the difficulty of adding a fourth task without moving the others.

**Paging.** Memory is divided into frames and each task receives frames as needed. Fragmentation is avoided. The control task's pages can be locked, so no fault can occur in its loop. The cost is a translation step, which is small.

**Segmentation.** Each task's code, data and stack are separate segments. This gives protection between tasks but adds little over paging for this design, and the segment table is unusual on Linux.

**Demand paging.** Pages are loaded when first touched. For the control task this is unacceptable, because the first touch of a page is a fault. For the logging task it is acceptable. With no swap, demand paging for anonymous memory brings no benefit, so the machine-learning task should have its memory allocated and touched at start-up.

**Table of choices.**

| Task | Mechanism | Justification |
|---|---|---|
| Real-time control | Paging with mlockall() after warm-up | Locked pages are never evicted; no faults in the control loop |
| Machine learning | Paging, memory pre-touched at start | Large region; pre-touch avoids first-touch faults |
| Logging | Demand paging with a bounded buffer | Tolerates faults; bounded by the buffer limit |
| Whole system | Paging, no swap, OOM protection | Avoids fragmentation; keeps total under the limit |

**Locking the control task's memory.** Call `mlockall(MCL_CURRENT | MCL_FUTURE)` at start-up, after the program has touched all of its code, data and stack, or use `MCL_CURRENT` only after warm-up. `MCL_CURRENT` locks pages that are already mapped; `MCL_FUTURE` locks later mappings, including the heap. Locking requires the `CAP_IPC_LOCK` capability or a sufficient `RLIMIT_MEMLOCK`. The lock ensures the pages stay resident, so the control loop sees no faults.

## 7.4 Stage 3: Implement and verify

**Locking program.** Appendix A.5 contains a program that runs a fixed computation one million times, records the worst iteration time, and accepts an argument that enables `mlockall`. The program also prints its fault counts.

**Measurement procedure.**

1. Build the program with optimisation disabled, so the loop is not removed: `gcc -O0 -o rt_loop rt_loop.c`.
1. Run it alone and record the worst iteration time and fault counts.
2. Start a competing memory-heavy process that touches 1.5 GB in a loop. The machine must have enough free RAM for this to cause pressure; on a larger machine, restrict the competitor with `systemd-run --scope -p MemoryMax=1G` or use `cgroups`.
3. Run the program without locking, then with locking, while the competitor runs.
4. Record the worst iteration time and the major and minor fault counts for each run.

The expected result is that the unlocked run shows a worst-case time far larger than the median, with non-zero fault counts, and the locked run shows a worst case close to the median with zero major faults after warm-up.

**Simulating the three tasks.** Record the reference stream of each task, or model it with a synthetic string that has the task's locality. Run the simulator in Appendix A.4 with a total budget equal to 2 GB minus the kernel reserve, and check that the total number of frames the three tasks need does not exceed the budget. This is the simulated check of R4.

**Verifying that the control task is never delayed.** The control task's worst iteration time, with locking, must be below its deadline with margin. Record the worst value over at least one million iterations and show it in the report.

## 7.5 Stage 4: Defend the design

**Justification from measurements.** The locked run shows a worst-case iteration time close to the median, with no faults. The unlocked run does not. This is the evidence that locking is necessary for R1. The simulation shows whether the three tasks fit within R4.

**A fourth task of 500 MB.** The committed total becomes 1.5 GB (ML) + 0.5 GB (new task) = 2.0 GB before the kernel, the control task, and the logging task are counted. The budget is exceeded by the kernel reserve and the other tasks, so R4 fails. The design must change: reduce the machine-learning footprint, move the new task to a larger device, or stream the new task's data so its resident set is smaller. Adding the task to this hardware is not acceptable.

**One remaining limitation.** Locking prevents page faults but not all sources of delay. The kernel can still delay the control task through interrupts, scheduling preemption, and cache effects, and Linux is not a hard real-time system. The worst-case latency is therefore bounded only statistically, from measurements. A PREEMPT_RT kernel reduces these delays, and the design should be validated on that kernel for a hard guarantee.

## 7.6 Alternative concept: a real-time OS with no virtual memory

Consider the same device with a real-time OS that has no virtual memory and uses a fixed partition for each task.

**Worst-case delay.** Without virtual memory there are no page faults and no translation misses. The worst-case delay is set by the scheduler and the interrupt latency, which RTOS

kernels bound tightly and in documented ways. This is a stronger guarantee than Linux with locking.

**Memory for the machine-learning task.** The ML task gets its fixed partition, typically the full budget left after the RTOS and the control task. There is no paging overhead and no kernel memory for virtual-memory structures, so more memory is available to the ML task than on Linux, where the kernel and its page tables use part of the 2 GB.

**Effort of adding tasks.** Each task must have its memory partition defined at build time. A new task needs its partition planned, the linker script updated, and the system rebuilt and retested. Linux with mlock() lets a new task start with `mlockall` and the memory budget checked at run time. The RTOS gives a stronger timing guarantee at the cost of more engineering effort for each change.

**Choice.** For the smart factory, where the control task's deadline is hard and the device is fixed in function, the RTOS is the stronger design. If the vendor expects frequent additions, Linux with a PREEMPT_RT kernel and locked memory is the more flexible design, provided the timing measurements are repeated on each new task.

## Conclusion to the Assignment

The seven questions form one argument. Linking choices determine how memory is shared between programs (Q1). Paging removes fragmentation and sets the cost of every address (Q2), while segmentation's logical structure survives only as virtual regions (Q3). Demand paging lets programs exceed RAM when their working sets fit, and thrashing marks the point where they do not (Q4). The page-replacement policy decides how well the working set is kept (Q5), storage configuration decides whether the system survives failures and boots (Q6), and a design for a constrained device combines locking, budgets and measurement (Q7).

The recurring theme is that every decision has a measurable cost: a fault rate, a table size, a rebuild window, a worst-case delay. The report recommends a choice in each case and states the measurement that supports it. Each recommendation should be checked against the team's own measurements before submission.

## Appendix A: Source Code

### A.1 Dynamic loading with dlopen (Q1)

The shared library `enc.c` provides a simple stand-in for the encryption routine:

```c
/* enc.c -- build with: gcc -shared -fPIC enc.c -o libenc.so */
int encrypt(int x)
{
    return x * 31 + 7;
}
```

The program `dl_demo.c` loads it at run time:

```c
/* dl_demo.c -- build with: gcc dl_demo.c -o dl_demo -ldl */
#include <stdio.h>
#include <dlfcn.h>

int main(void)
{
    void *h = dlopen("./libenc.so", RTLD_NOW);
    if (h == NULL) {
        fprintf(stderr, "dlopen failed: %s\n", dlerror());
        return 1;
    }

    int (*encrypt)(int) = (int (*)(int)) dlsym(h, "encrypt");
    if (encrypt == NULL) {
        fprintf(stderr, "dlsym failed: %s\n", dlerror());
        dlclose(h);
        return 1;
    }

    printf("encrypt(7) = %d\n", encrypt(7));
    dlclose(h);
    return 0;
}
```

Changing `x * 31 + 7` to `x * 31 + 9`, rebuilding only `libenc.so`, and running `dl_demo` again shows the new output with the same executable. This is the "replace the library without recompiling" experiment.

## A.2 Address translation (Q2)

```c
/* translate.c -- logical to physical translation with a one-level page table
*/
#include <stdio.h>
#include <stdint.h>

#define PAGE_SIZE 4096u

int main(void)
{
    /* page_table[p] = frame number for page p; the example maps page 2 to
frame 7 */
    uint32_t page_table[] = {5, 9, 7, 3};
    uint32_t num_pages = sizeof(page_table) / sizeof(page_table[0]);

    uint32_t logical[] = {0, 4095, 4096, 8200, 16383, 16384};
    for (unsigned i = 0; i < sizeof(logical) / sizeof(logical[0]); i++) {
        uint32_t addr = logical[i];
        uint32_t p = addr / PAGE_SIZE;
```

```
        uint32_t d = addr % PAGE_SIZE;

        if (p >= num_pages) {
            printf("logical %u: page %u out of range -> invalid address\n",
addr, p);
            continue;
        }
        uint32_t frame = page_table[p];
        printf("logical %u: page %u offset %u frame %u physical %u\n",
               addr, p, d, frame, frame * PAGE_SIZE + d);
    }
    return 0;
}
```

The output for logical address 8200 is page 2, offset 8, frame 7, physical 28680. The addresses 16383 and 16384 show the boundary: the last byte of page 3 is valid, and the first byte of the nonexistent page 4 is rejected.

### A.3 Segment translation (Q3)

```
/* segtrans.c -- (segment, offset) to physical address with base and limit */
#include <stdio.h>

struct segment { unsigned base, limit; };

static struct segment table[] = {
    {1400, 1000},   /* 0: code  */
    {4300, 1100},   /* 1: data  */
    {6300,  400},   /* 2: stack */
};

int translate(unsigned s, unsigned d, unsigned *phys)
{
    if (s >= sizeof(table) / sizeof(table[0]))
        return -1;                      /* no such segment */
    if (d >= table[s].limit)
        return -2;                      /* trap: offset beyond limit */
    *phys = table[s].base + d;
    return 0;
}

int main(void)
{
    unsigned tests[][2] = {
        {0, 53}, {2, 399}, {2, 400}, {1, 1099},
        {1, 1100}, {0, 999}, {0, 1000}, {2, 0}
    };
    for (unsigned i = 0; i < sizeof(tests) / sizeof(tests[0]); i++) {
        unsigned phys;
        int r = translate(tests[i][0], tests[i][1], &phys);
```

```
        if (r == 0)
            printf("(%u, %u) -> %u\n", tests[i][0], tests[i][1], phys);
        else
            printf("(%u, %u) -> TRAP (%s)\n", tests[i][0], tests[i][1],
                   r == -2 ? "offset beyond limit" : "no such segment");
    }
    return 0;
}
```

The eight results are 1453, 6699, trap, 5399, trap, 2399, trap and 6300, matching section 3.3.

## A.4 Page-replacement simulator (Q5)

```
# sim.py -- FIFO, LRU and Optimal fault counters
def fifo(refs, n):
    frames, faults = [], 0
    for p in refs:
        if p not in frames:
            faults += 1
            if len(frames) == n:
                frames.pop(0)
            frames.append(p)
    return faults

def lru(refs, n):
    frames, faults = [], 0          # index 0 is least recently used
    for p in refs:
        if p in frames:
            frames.remove(p)
        else:
            faults += 1
            if len(frames) == n:
                frames.pop(0)
        frames.append(p)
    return faults

def optimal(refs, n):
    frames, faults = [], 0
    for i, p in enumerate(refs):
        if p in frames:
            continue
        faults += 1
        if len(frames) < n:
            frames.append(p)
            continue
        def next_use(f):
            try:
                return refs.index(f, i + 1)
            except ValueError:
                return float("inf")
```

```
        victim = max(frames, key=next_use)
        frames[frames.index(victim)] = p
    return faults

def random_string(pages, length, seed):
    import random
    rng = random.Random(seed)
    return [rng.randrange(pages) for _ in range(length)]

def locality_string(length, seed, hot=10, total=50, p_hot=0.8):
    import random
    rng = random.Random(seed)
    out = []
    for _ in range(length):
        if rng.random() < p_hot:
            out.append(rng.randrange(hot))
        else:
            out.append(rng.randrange(total))
    return out

if __name__ == "__main__":
    belady = [1,2,3,4,1,2,5,1,2,3,4,5]
    for n in (3, 4):
        print(n, "frames:", "FIFO", fifo(belady, n),
              "LRU", lru(belady, n), "OPT", optimal(belady, n))
```

Running the file prints the Belady results from Table 5.1. The functions `locality_string` and `random_string` produce the 100,000-reference strings for the experiment.

## A.5 Locked real-time loop (Q7)

```
/* rt_loop.c -- worst-case iteration time, with optional mlockall()
   build:  gcc -O0 -o rt_loop rt_loop.c
   run:    ./rt_loop            (no locking)
           ./rt_loop lock       (mlockall, needs CAP_IPC_LOCK or sudo)
*/
#define _GNU_SOURCE
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <sys/mman.h>
#include <sys/resource.h>

static double now_ns(void)
{
    struct timespec t;
    clock_gettime(CLOCK_MONOTONIC, &t);
    return t.tv_sec * 1e9 + t.tv_nsec;
}
```

```c
int main(int argc, char **argv)
{
    int lock = (argc > 1 && strcmp(argv[1], "lock") == 0);

    /* touch a working buffer first so it is resident before locking */
    static double work[4096];
    for (int i = 0; i < 4096; i++) work[i] = i;

    if (lock && mlockall(MCL_CURRENT | MCL_FUTURE) != 0) {
        perror("mlockall");
        return 1;
    }

    volatile double acc = 0.0;
    double worst = 0.0, total = 0.0;
    const int iters = 1000000;

    for (int i = 0; i < iters; i++) {
        double t0 = now_ns();
        for (int k = 0; k < 100; k++)
            acc += work[(i + k) & 4095] * 0.5;
        double dt = now_ns() - t0;
        total += dt;
        if (dt > worst) worst = dt;
    }

    struct rusage ru;
    getrusage(RUSAGE_SELF, &ru);
    printf("mode=%s mean=%.0f ns worst=%.0f ns minor_faults=%ld
major_faults=%ld\n",
           lock ? "locked" : "unlocked", total / iters, worst,
           ru.ru_minflt, ru.ru_majflt);
    return 0;
}
```

The program reports the mean and worst iteration times and the fault counts. Running it with and without `lock` while a memory-heavy competitor runs gives the comparison described in section 7.4. Expect the locked mode to show the smaller worst-case value and no major faults after warm-up.

## Appendix B: Verification Traces (Q5)

Belady string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5. Each row shows the frame contents after the reference, and F marks a fault.

**FIFO, 3 frames.** 1 F [1]; 2 F [1 2]; 3 F [1 2 3]; 4 F [2 3 4]; 1 F [3 4 1]; 2 F [4 1 2]; 5 F [1 2 5]; 1 hit; 2 hit; 3 F [2 5 3]; 4 F [5 3 4]; 5 hit. Total faults: **9**.

**FIFO, 4 frames.** 1 F [1]; 2 F [1 2]; 3 F [1 2 3]; 4 F [1 2 3 4]; 1 hit; 2 hit; 5 F [2 3 4 5]; 1 F [3 4 5 1]; 2 F [4 5 1 2]; 3 F [5 1 2 3]; 4 F [1 2 3 4]; 5 F [2 3 4 5]. Total faults: **10**.

**LRU, 3 frames.** 1 F [1]; 2 F [1 2]; 3 F [1 2 3]; 4 F [2 3 4]; 1 F [3 4 1]; 2 F [4 1 2]; 5 F [1 2 5]; 1 hit; 2 hit; 3 F [1 2 3]; 4 F [2 3 4]; 5 F [3 4 5]. Total faults: **10**.

**LRU, 4 frames.** 1 F; 2 F; 3 F; 4 F [1 2 3 4]; 1 hit; 2 hit; 5 F, evict 3 → [4 1 2 5]; 1 hit; 2 hit; 3 F, evict 4 → [5 1 2 3]; 4 F, evict 5 → [1 2 3 4]; 5 F, evict 1 → [2 3 4 5]. Total faults: **8**.

**Optimal, 3 frames.** 1 F; 2 F; 3 F [1 2 3]; 4 F, evict the page used farthest ahead (3) → [1 2 4]; 1 hit; 2 hit; 5 F, evict 4 → [1 2 5]; 1 hit; 2 hit; 3 F, evict 1 (never used again) → [3 2 5]; 4 F, evict 3 (never used again) → [4 2 5]; 5 hit. Total faults: **7**.

**Optimal, 4 frames.** 1 F; 2 F; 3 F; 4 F [1 2 3 4]; 1 hit; 2 hit; 5 F, evict 4 → [1 2 3 5]; 1 hit; 2 hit; 3 hit; 4 F, evict 1 (never used again) → [4 2 3 5]; 5 hit. Total faults: **6**.

These match the simulator output in section 5.3.

## Appendix C: Command Reference

| Question | Command | Purpose |
|---|---|---|
| Q1 | `gcc -static bank.c -L. -lenc -o bank_static` | Build static executable |
| Q1 | `gcc bank.c -L. -lenc -Wl,-rpath,'$ORIGIN' -o bank_dyn` | Build dynamic executable |
| Q1 | `ls -l bank_static bank_dyn; size bank_static bank_dyn` | Compare sizes |
| Q1 | `ldd bank_dyn` | List shared libraries |
| Q1 | `cat /proc/self/maps` (inside the program) | Show load addresses |
| Q1 | `perf stat -r 100 ./bank_dyn` | Measure start-up time |
| Q2 | `getconf PAGESIZE` | Linux page size |
| Q2 | `/usr/bin/time -v ./prog` | Page faults and memory |
| Q3 | `cat /proc/<pid>/maps` | Segments of a running process |
| Q4 | `grep VmRSS /proc/self/status` | Resident memory before and after writes |
| Q6 | `lsblk`, `sudo fdisk -l` | Partitions |
| Q6 | `swapon --show`, `free -h` | Swap in use |
| Q7 | `./rt_loop` and `./rt_loop lock` | Worst-case timing with and without locking |